

Sri Lanka Institute of Information Technology

Programming Applications and Frameworks (IT3030)

Final Assignment Report

GROUP ID: Y3S1_WD_12



Smart Campus

Smart Campus Operations Hub

Student ID	Student Name
IT23570108	WIJESINGHE A G T N
IT23760356	KAVINDA K A L H G
IT23656260	PRASHADIKA D P D K
IT23591202	NETHMINI S A T

Date: 30.04.2026

Table of Contents

1. Introduction 3

2. Functional Requirements & Non-Functional Requirements 4

3. Overall Architecture Diagram..... 10

4. REST API Architecture diagram 11

5. Frontend Architecture Diagram 12

6. System Functions 13

7. GitHub..... 30

8. References **Error! Bookmark not defined.**

1. Introduction

The Smart Campus Management System is a cutting-edge, full-stack enterprise application that aims to streamline resource and student services management in universities. With the increasing complexity of educational institutions, there is a dire need for an integrated solution to manage booking of resources, events and instant communication. Our project meets these needs by creating a digital ecosystem for student, faculty and administration interaction.

The system is developed using a high-performance architecture and is split into a number of modules. The Booking Management Module is responsible for managing the scheduling of lecture rooms, labs and equipment with intelligent conflict resolution and capacity checks. For secure and convenient access, the system uses Google OAuth2 authentication, enabling users to securely login with their university ID.

It also uses a Smart Notification System for updates on request statuses and an Admin Dashboard with bulk operations and analytics for insights. Using a modern technology stack, including React.js for the frontend and Spring Boot with PostgreSQL for the backend, the Smart Campus Management System offers a robust system that improves transparency, simplifies administration and maximises the use of campus resources.

This version adds details about:

- Challenges: Why the system is needed (growing complexity).
- Security: Google OAuth2 and University credentials.
- Efficiency: Notify system and Admin Analytics.
- Tech Stack: Specifically mentions React, Spring Boot and PostgreSQL.

2. Functional Requirements & Non-Functional Requirements

IT23591202 - Facilities catalogue & resource management

Functional Requirements

1. **Resource Categorization:** The system shall allow administrators to categorize resources into types such as Lecture Halls, Laboratories, Meeting Rooms, and Equipment.
2. **Resource CRUD Operations:** Administrators shall be able to Create, Read, Update, and Delete resource records, including details like Name, Location, and Description.
3. **Capacity Management:** Each resource must have a defined maximum capacity to ensure booking requests do not exceed physical limits.
4. **Availability Configuration:** The system shall allow setting operating hours (Open/Close times) for each resource to prevent bookings outside of working hours.
5. **Image Integration:** The system shall support uploading and displaying images for each resource to help users identify the location.
6. **Real-time Availability Status:** The system shall dynamically show whether a resource is "Available" or "Booked" based on the current time and existing approved bookings.

Non-Functional Requirements

1. **Consistency:** Any update to a resource (e.g., a change in capacity) must be immediately reflected in all pending and future booking validations.
2. **Accuracy:** Resource data, especially availability windows, must be 100% accurate to prevent scheduling errors.
3. **Visual Performance:** Resource images must be optimized and served via a CDN or efficient file handling

to ensure fast page load times on the frontend.

4. **Extensibility:** The resource schema should be flexible enough to add new types of assets (e.g., parking slots or sports fields) in the future without major code changes.
5. **Data Security:** Only users with high-level administrative privileges (ADMIN role) shall be allowed to modify resource configurations.

IT23656260 – Incident Tickets & Technician Updates

Functional Requirements

1. **Facility Reservation:** Users shall be able to book a specific facility for a selected date and time slot.
2. **Availability Calendar:** The system shall provide a real-time calendar view showing all existing bookings and available slots.
3. **Booking Confirmations:** The system shall automatically send an email confirmation to the user upon a successful booking.
4. **Manage Bookings:** Users shall be able to view, cancel, or reschedule their own bookings through a personal dashboard.
5. **AI Chatbot Assistance:** The system shall include a chatbot to answer frequently asked questions about campus services and facilities.
6. **Conversational Booking:** The chatbot shall be capable of initiating a booking process through natural language interaction.
7. **Booking Conflicts:** The system shall automatically detect and prevent overlapping booking requests for the same slot.

Non-Functional Requirements

1. **Concurrency Control:** The system must ensure that two users cannot book the same facility for the same time slot simultaneously.
2. **Response Time:** The AI chatbot must provide responses to user inquiries within 1 second for a

seamless experience.

3. **Privacy:** User booking data must be kept private and only accessible to the account owner and authorized staff.
4. **Chatbot Accuracy:** The chatbot should achieve at least an 85% accuracy rate in recognizing user intents.
5. **High Availability:** The booking engine must be accessible 24/7, maintaining a minimum uptime of 99.9%.
6. **Robustness:** The system should gracefully handle invalid inputs in the chatbot or booking forms without crashing.
7. **State Persistence:** If a user's session is interrupted during a booking, the system should allow them to resume where they left off.

IT23760356 – Booking Management

Functional Requirements

1. **Facility Booking:** Users shall be able to request bookings for specific resources (rooms, labs, halls) by selecting a date and time slot.
2. **Conflict Detection:** The system must automatically detect and prevent overlapping bookings for the same resource to avoid double-booking.
3. **Real-time Validation:** The system shall validate that booking dates are not in the past and that the requested capacity does not exceed the resource limit.
4. **Booking Management:** Users shall be able to view, edit, or cancel their own PENDING bookings.
5. **Admin Dashboard:** Administrators shall be able to view all bookings and filter them by date, resource type, or status.
6. **Approval Workflow:** Administrators shall have the authority to Approve or Reject booking requests with optional feedback/reasons.
7. **Bulk Processing:** The system shall allow administrators to process multiple booking requests simultaneously (Bulk Approve/Reject).

8. **QR Verification:** The system shall provide a verification endpoint to validate bookings via QR code scanning on the day of the event.

Non-Functional Requirements

1. **Security:** All booking actions must be secured using JWT tokens, ensuring that only authenticated users can make requests and only Admins can access management features.
2. **Data Integrity:** The system must ensure that once a time slot is approved, it is locked in the database to prevent any data inconsistencies.
3. **Performance:** The conflict detection query and API responses should be optimized to return results in under 200ms.
4. **Usability:** The booking interface must be responsive and intuitive, allowing users to complete a booking request in fewer than 4 clicks.
5. **Reliability:** The system should maintain a 99.9% uptime for the booking service to ensure students can make requests at any time.
6. **Scalability:** The backend must be designed to handle a high volume of concurrent booking requests during peak university hours (e.g., at the start of a semester)

IT23570108- Authentication/security and Notification Management

Functional Requirements

1. **Authentication & User Login:** The system shall allow users to log in using Google OAuth2 without requiring a password.
2. **User Account Management:** The system shall automatically create a new user account if the email is not registered and update existing user details (name, profile picture) on each login.
3. **JWT Token Generation:** The system shall generate a JWT token after successful login to authenticate future API requests.
4. **User Profile Access:** The system shall allow users to view their profile information via a secured endpoint.

5. **User Role Management:** The system shall allow administrators to view all users and update user roles such as USER, ADMIN, and TECHNICIAN.
6. **Role-Based Access Control:** The system shall restrict access to API endpoints based on user roles to ensure authorized usage.
7. **Secure API Communication:** The system shall validate JWT tokens for every request using authorization headers.
8. **Notification Creation:** The system shall generate notifications when important events occur such as booking approval, booking rejection, ticket assignment, and ticket status updates.
9. **View Notifications:** The system shall allow users to view all notifications and unread notifications.
10. **Notification Status Management:** The system shall allow users to mark individual notifications or all notifications as read.
11. **Unread Notification Count:** The system shall display the number of unread notifications to users as a badge indicator.
12. **User-Specific Notifications:** The system shall ensure that each notification is linked to a specific user.
13. **Notification Preferences:** The system shall allow users to enable or disable email and push notifications.

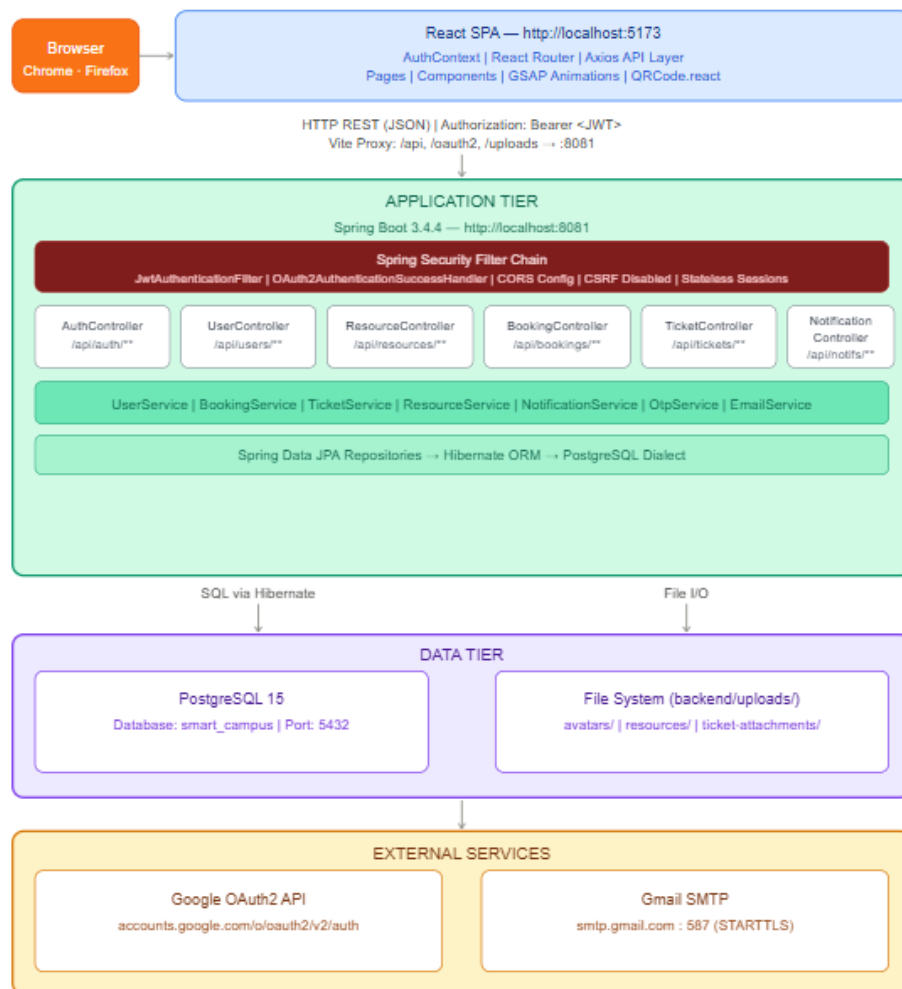
Non-Functional Requirements

1. **Security:** The system shall ensure secure authentication using OAuth2 and JWT while protecting sensitive data and preventing unauthorized access.
2. **Performance:** The system shall provide fast API response times and efficient retrieval of notification data, including real-time badge updates.
3. **Scalability:** The system shall support multiple concurrent users and handle increasing volumes of notifications efficiently.
4. **Maintainability:** The system shall follow a modular architecture (Controller, Service, Repository) to allow easy updates and maintenance.
5. **Usability:** The system shall provide a simple login process and an intuitive user interface with clear notification indicators.
6. **Reliability:** The system shall ensure consistent operation during login and notification delivery without data loss.

7. **Compatibility:** The system shall be compatible with a React frontend and REST API backend, and support cross-origin requests (CORS).

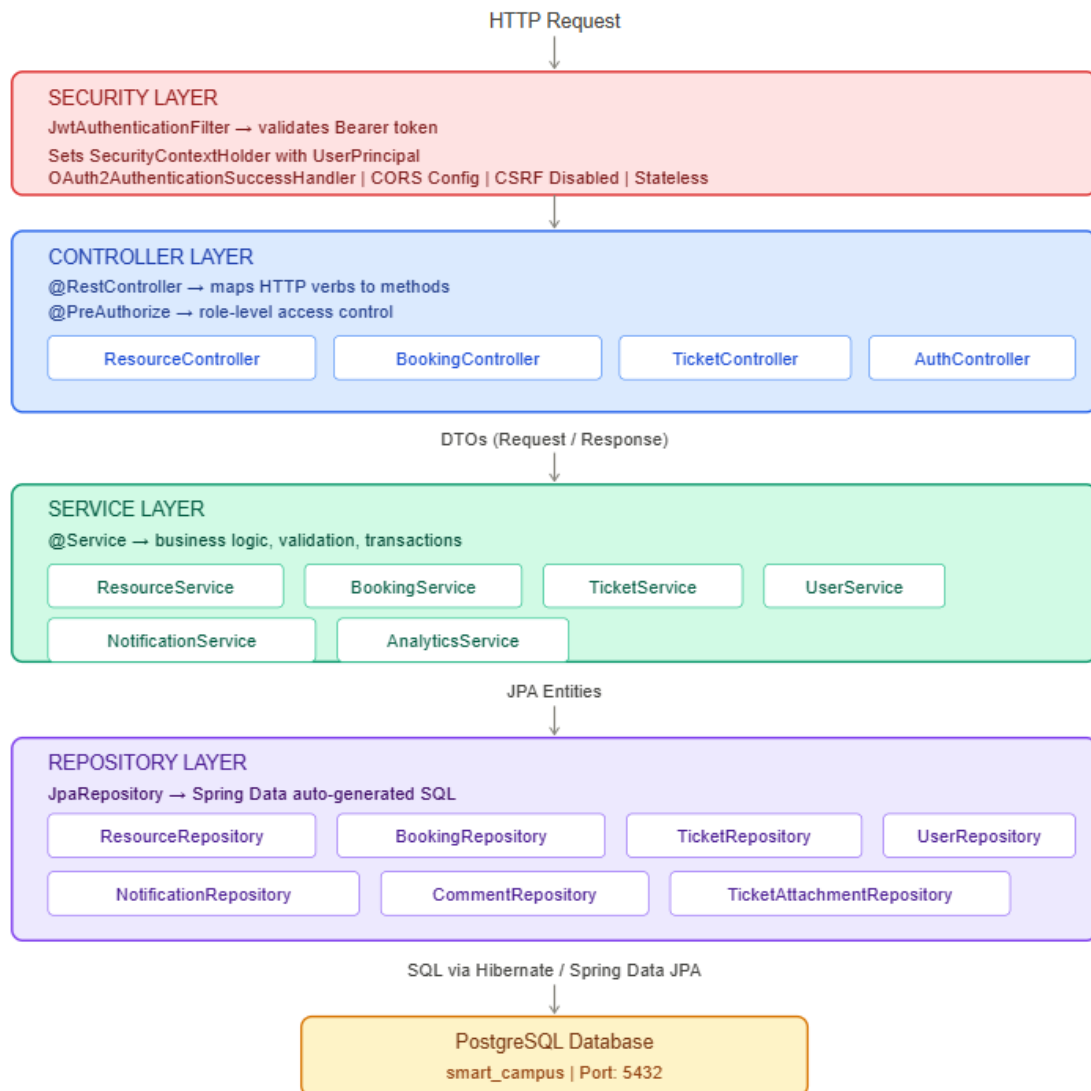
3. Overall Architecture Diagram

- The system follows a client-server architecture with a clear separation between the React frontend and the Spring Boot backend. Communication is exclusively over HTTP using JSON payloads and JWT Bearer tokens for authentication



4. REST API Architecture diagram

- The backend follows a layered architecture pattern with strict separation of concerns across four layers:



5. Frontend Architecture Diagram

- The frontend is a React Single-Page Application using React Router v7 for client-side routing and a centralised Auth text for authentication state management across all pages



- Route Guard Pattern: App.jsx wraps protected routes in ProtectedRoute which checks AuthContext. If the user is not authenticated they are redirected to /login. If the route requires a specific role (e.g. ADMIN), users with insufficient roles receive a 403 message

6. System Functions

- Description of each member's function and screenshots of the UI.

Member 1: Booking workflow & conflict checking

Student ID & Name: IT23760356 – KAVINDA K A L H G

Description:

Developed the booking request system, the approval/rejection workflow, and the critical backend logic required to prevent scheduling conflicts.

Special Features:

My module includes a **Smart QR Verification System** that automates user check-ins by validating booking details in real-time through a scanner, and an **Administrative Bulk Action** feature that allows managers to approve or reject multiple requests simultaneously to save time and improve efficiency.

Endpoint Descriptions:

- **POST /api/bookings:** Create a facility booking with backend validation for past dates, capacity limits, and conflict detection.
- **GET /api/bookings/my:** Retrieve a list of bookings belonging to the currently authenticated user.
- **GET /api/bookings/{id}:** Retrieve detailed information for a specific booking by its unique ID.
- **PATCH /api/bookings/{id}/cancel:** Allows the owner to cancel a PENDING booking by providing a cancellation reason.
- **GET /api/bookings:** Administrative search and filtering of all bookings by **Resource**, **Date**, and **Status**.
- **GET /api/bookings/resource/{id}:** List all approved bookings for a specific resource to show its current schedule.
- **PATCH /api/bookings/{id}/action:** Allows Admins/Managers to Approve or Reject a booking (with a rejection reason).
- **PATCH /api/bookings/bulk-action:** **[New]** Allows Admins to Approve or Reject multiple bookings at once for better efficiency.
- **PATCH /api/bookings/{id}/verify:** **[New]** Verification endpoint for the QR Scanner system to check-in users on the day of the booking.

User side:

Booking Dashboard

 **Smart Campus**

Resources

My Bookings

Tickets

 7

 gihan

USER



My Bookings

Manage your facility reservations at Smart Campus

 **7**
TOTAL

 **6**
APPROVED

 **1**
PENDING

+ New Booking

Upcoming 7

Completed

History

UPCOMING — MAY 2026



Lecture Hall F1103
2026-05-01 08:00:00 – 10:00:00 Block F , New Building
For Meeting

APPROVED

 QR

 Cancel



Lecture Hall F1103

All Pervious Booking:

 **Smart Campus**

Resources

My Bookings

Tickets

 7

 gihan

USER





Lecture Hall F1103
2026-05-01 08:00:00 – 10:00:00 Block F , New Building
For Meeting

APPROVED

 QR

 Cancel



Lecture Hall F1103
2026-05-01 10:00:00 – 18:00:00 Block F , New Building
For VIVA

APPROVED

 QR

 Cancel



Lab New F505
2026-05-02 13:00:00 – 17:00:00 New Building
For DS Lecture

APPROVED

 QR

 Cancel



Lecture Hall F1103
2026-05-03 09:00:00 – 10:00:00 Block F , New Building
For meeting

APPROVED

 QR

 Cancel



Lecture Hall F1103
2026-05-03 11:00:00 – 14:00:00 Block F , New Building
For lecture

APPROVED

 QR

 Cancel



Lab New F505
2026-05-01 14:00:00 – 16:00:00 New Building
For Lab

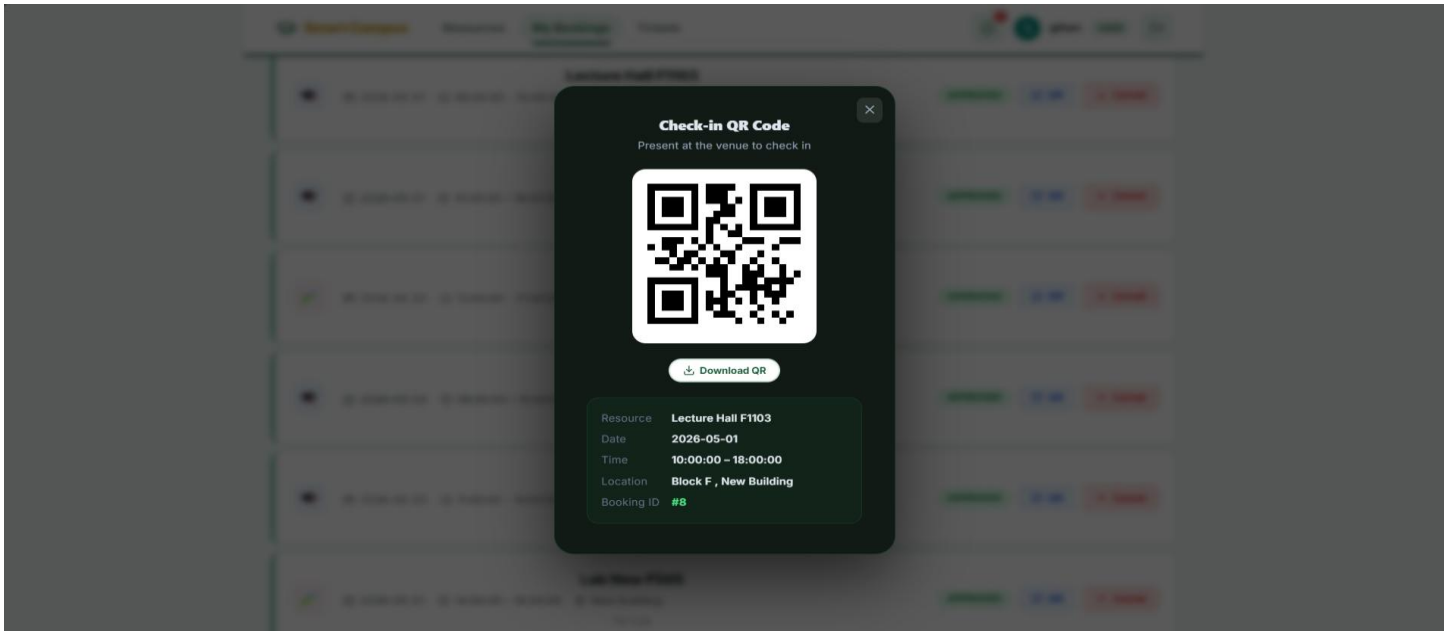
APPROVED

 QR

 Cancel

Page 14 | 33

Example QR:



Admin side:

Admin Booking Dashboard:

Smart Campus

ResourcesMy BookingsTicketsAdmin

HGADMIN

NAVIGATION

Bookings

Users

Resources

Analytics

RESOURCE: All ResourcesDATE: dd/mm/yyyyReset

ALLPENDINGAPPROVEDREJECTEDCANCELLEDVERIFIEDScan QR

UPCOMING — MAY 2026

Lecture Hall F1103

by gihan kavinda

2026-05-03

11:00:00 – 14:00:00

For lecture

APPROVED

Lab New F505

by gihan kavinda

2026-05-01

14:00:00 – 16:00:00

For Lab

APPROVED

Lecture Hall F1103

by gihan kavinda

2026-05-03

09:00:00 – 10:00:00

For meeting

APPROVED

Lab New F505

by HG Kavinda

2026-05-02

17:00:00 – 18:00:00

For short meeting

APPROVED

Lab New F505

by gihan kavinda

2026-05-02

13:00:00 – 17:00:00

APPROVED

Admin QR Scanner:

← Admin Panel

QR Check-in Scanner

Point camera at QR or upload a saved image to verify

Camera Scanner

Use your device camera for real-time check-in

Open Camera

Scan Image File

Upload a QR code image from your gallery

Browse Files

Page 16 | 33

IT23591202 - Facilities catalogue & resource management

Description:

Implemented the catalogue for bookable resources, supporting metadata tracking and search/filtering capabilities by type, capacity, and location.

Endpoint Descriptions:

GET/api/resources: List all campus resources with optional type, status, location, and minimum-capacity filters

GET /api/resources/{id}:Retrieve a single resource by ID

POST/api/resources: Create a new resource (Admin/Manager only)

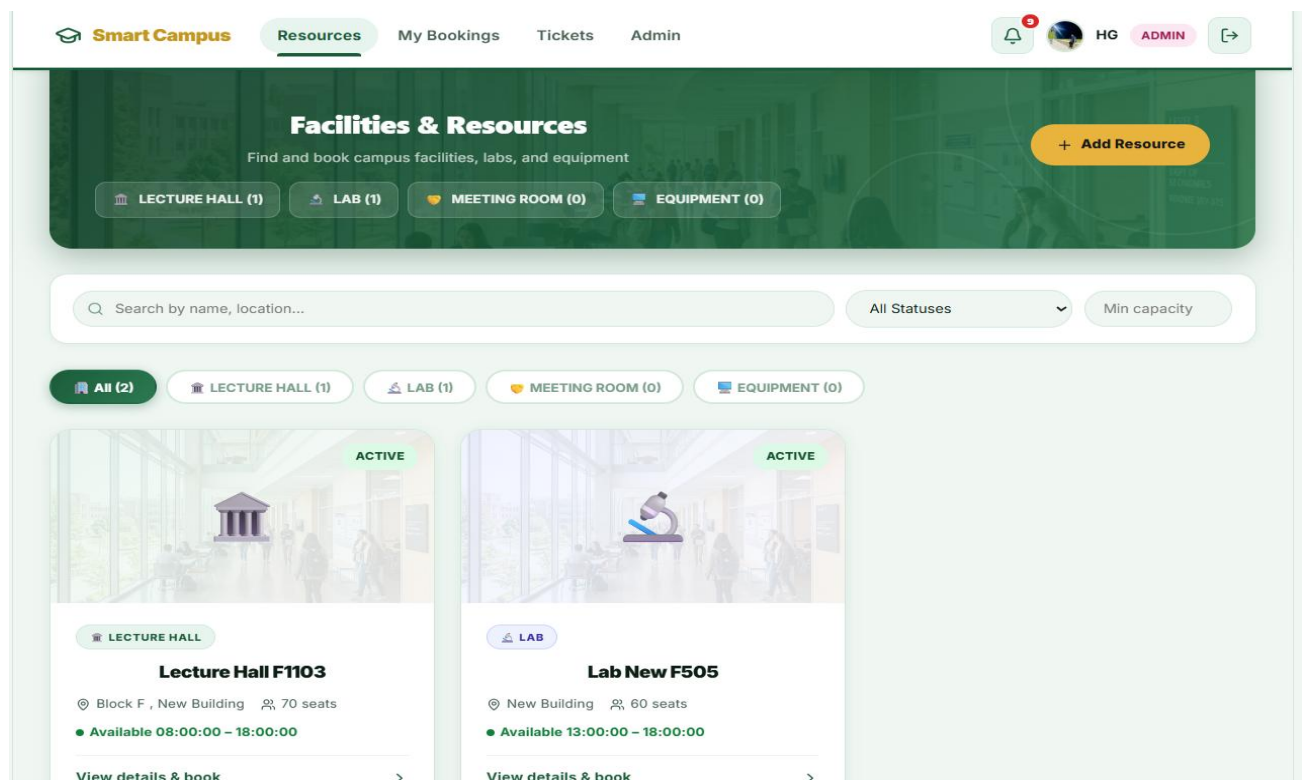
PUT/api/resources/{id}:Update an existing resource (Admin/Manager only)

DELETE/api/resources/{id}:Delete a resource (Admin/Manager only)

GET/api/analytics:Retrieve aggregated analytics: top resources, peak hours, booking counts (Admin/Manager)

User side :

Recourses Dashboard:



The screenshot shows the 'Smart Campus' interface with the 'Resources' tab selected. The main header includes a navigation bar with 'Smart Campus', 'Resources', 'My Bookings', and 'Tickets'. A user profile for 'Pradeep' is visible in the top right. The main content area features a large banner for 'Lab 403' with a background image of a network diagram and a 'Lab 403' title. Below the banner, there are two columns of information:

- Resource Information:**
 - LOCATION:** Block A
 - CAPACITY:** 20 persons
 - AVAILABILITY:** 11:17:00 - 01:17:00
 - TYPE:** Lecture Hall
- Details:**
 - Status:** Active
 - Type:** Lecture Hall
 - Capacity:** 20 persons
 - Location:** Block A
 - Open Hours:** 11:17:00-01:17:00

Below these columns, there is an 'About This Resource' section with a placeholder text 'hello'. At the bottom, there is a 'Book This Resource' button with a note 'Requests require admin approval'.

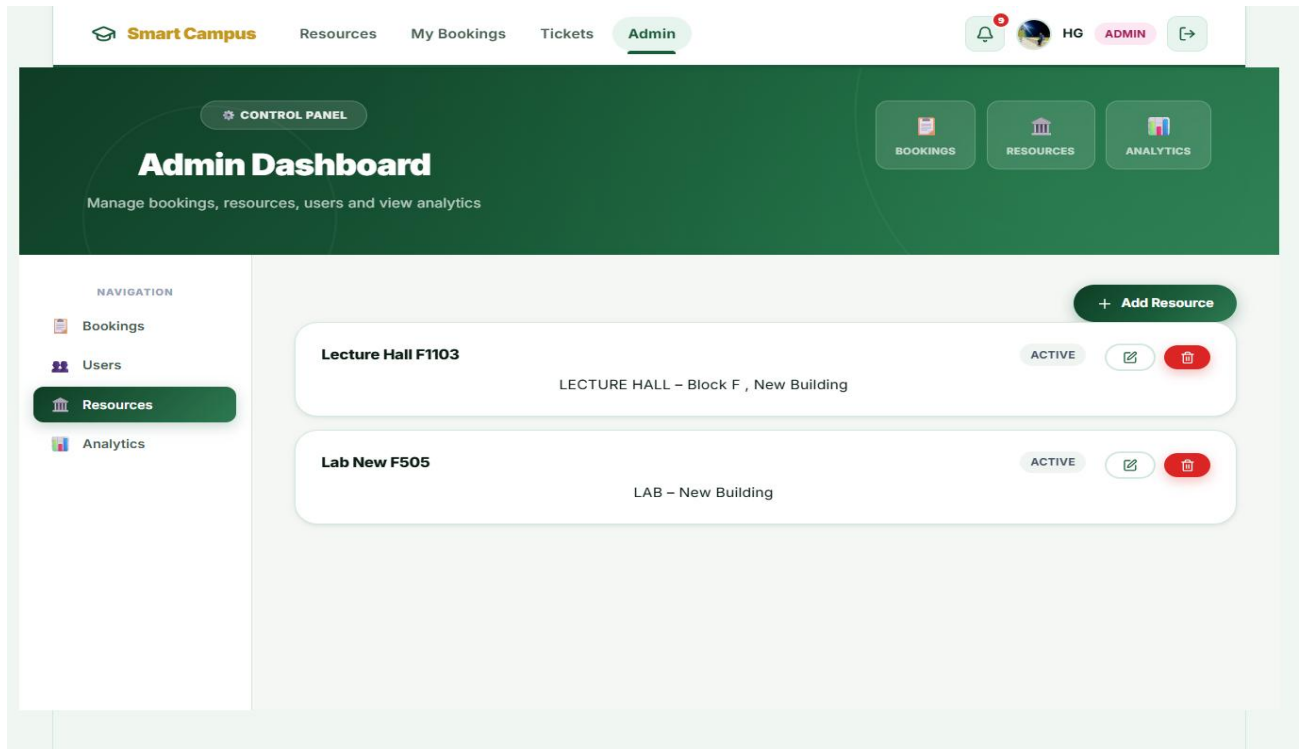
Admin side:

Add resources Form

The screenshot shows the 'Smart Campus' interface with the 'Admin' tab selected. The main header includes a navigation bar with 'Smart Campus', 'Resources', 'My Bookings', 'Tickets', and 'Admin'. A user profile for 'HG ADMIN' is visible in the top right. The main content area features a large form for adding resources:

- Name ***: Text input field with placeholder 'e.g. Lab A301'.
- Type ***: Dropdown menu with 'LECTURE HALL' selected.
- Status ***: Dropdown menu with 'ACTIVE' selected.
- Location ***: Text input field with placeholder 'e.g. Block A, Floor 3'.
- Capacity**: Text input field.
- Available From**: Date and time picker.
- Available Until**: Date and time picker.
- Description**: Text area with placeholder 'Optional description...'.
- Image**: File upload section with a 'Choose File' button and 'No file chosen' text.
- Create Resource**: A large green button at the bottom.

Recourses Admin Dashboard :



IT23656260 - Incident tickets, attachments & technician updates

Descriptions: Member 3 implemented the full incident ticket lifecycle — submission with file attachments, threaded comments, technician assignment, status tracking, and the technician-specific dashboard.

Branches:

- feature/module-c-ticket-entity
- feature/module-c-ticket-ui
- feature/module-c-ticket-backend

Admin side:

Issue Details Form:

[← Back](#)[INCIDENT REPORTING](#)

Report a Campus Issue

Submit a maintenance request or campus incident report. Our facilities team will review and respond promptly.



Issue Details

Fill in the information below to submit your request

CATEGORY



Electrical



Plumbing



HVAC



IT Equipment



Furniture



Security



Cleaning



Other

PRIORITY LEVEL



Low

Not urgent — can wait



Medium

Should be fixed soon



High

Impacts daily operations



Critical

Immediate action required

Location

e.g. Block A, Room 301

Related Resource

None — General issue

**Low**

Not urgent — can wait

**Medium**

Should be fixed soon

**High**

Impacts daily operations

**Critical**

Immediate action required

Location *

e.g. Block A, Room 301

Related Resource

None — General issue

Description *

Describe the issue in detail. Include when it started, what the impact is, and any relevant context...

0/2000

Preferred Contact

Email or phone number for updates

Attachments (up to 3 images)**Click or drag images here**

JPG, PNG or GIF — max 3 files

Cancel

Submit Ticket

IT Equipment Report Page:

[Resources](#)[My Bookings](#)[Tickets](#)[Admin](#)

 **Dilukshi** ADMIN 

[← Back to Tickets](#)

TICKET #7

IT EQUIPMENT

IT EQUIPMENT Report

[Open](#) [Medium Priority](#)

 **Issue Details**

LOCATION

block a

REPORTER

Dilukshi Kaushalya

CONTACT

0741235461

DESCRIPTION

hiii

 **Attachments (1/3)**

9.jpg



[Add Image](#)

 **Status Overview**

Current Status

[Open](#)

Priority

[Medium](#)

Category

IT EQUIPMENT

 **Staff Update**

OPEN



Assign technician...



Resolution notes...

[Update Ticket](#)



Update Ticket

9.jpg

Add Image



Comments (3)

Dilukshi Kaushalya

4/29/2026, 8:33:57 PM

hii

Edit Delete

Dilukshi Kaushalya

4/29/2026, 8:34:05 PM

hey

Edit Delete

Dilukshi Kaushalya

4/29/2026, 8:34:16 PM

haiii

Edit Delete

Add a comment or note...

Post Comment

Endpoint :

- POST/api/tickets
- GET/api/tickets/my
- GET/api/tickets/{id}
- GET/api/tickets
- PATCH/api/tickets/{id}
- POST/api/tickets/{id}/comments
- PUT/api/tickets/{ticketId}/comments/{id}
- DELETE/api/tickets/{ticketId}/comments/{id}
- POST/api/tickets/{id}/attachments
- GET/api/tickets/{ticketId}/attachments/{id}
- DELETE/api/tickets/{ticketId}/attachments/{id}

Postmen testing:A screenshot of a Postman collection named 'smart_campus-m3'. The collection contains six tests, each with a colored icon indicating the HTTP method: POST (orange), GET (green), PATCH (blue), PATCH (blue), POST (orange), and POST (orange). The tests are: 'TEST 1 — Create Ticket (authenticated user — success)', 'TEST 2 — Get Ticket by ID (owner — success)', 'TEST 3 — Assign Ticket to Technician (ADMIN — success)', 'TEST 4 — Update Ticket Status (TECHNICIAN — success)', 'TEST 5 — Add Comment to Ticket (authenticated user — success)', and 'TEST 6 — Upload Attachment to Ticket'. The last test is highlighted with a grey background.

smart_campus-m3

POST TEST 1 — Create Ticket (authenticated user — success)

GET TEST 2 — Get Ticket by ID (owner — success)

PATCH TEST 3 — Assign Ticket to Technician (ADMIN — success)

PATCH TEST 4 — Update Ticket Status (TECHNICIAN — success)

POST TEST 5 — Add Comment to Ticket (authenticated user — success)

POST TEST 6 — Upload Attachment to Ticket

smart_campus-m3 > TEST 1 — Create Ticket (authenticated user — success)

POST http://localhost:8080/api/tickets Send

Docs Params Authorization Headers 11 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {"title": "Projector not working in Lab A101", "description": "The ceiling projector in Lab A101 fails to turn on.", "category": "IT_EQUIPMENT", "priority": "HIGH", "location": "Block A, Floor 1, Lab A101"}
```

Body Cookies 1 Headers 15 Test Results 3/4 201 Created • 457 ms • 1.05 KB Save Response

{ } JSON Preview Visualize

```
3 message: "Ticket created",
4 "data": {
5   "id": 1,
6   "resourceId": null,
7   "resourceName": null,
8   "reporterId": 2,
9   "reporterName": "Pradeep Sripal",
10  "assigneeId": null,
11  "assigneeName": null,
12  "category": "IT_EQUIPMENT",
13  "description": "The ceiling projector in Lab A101 fails to turn on.",
14  "priority": "HIGH",
15  "status": "OPEN",
16  "location": "Block A, Floor 1, Lab A101",
17  "preferredContact": null,
18  "rejectionReason": null,
19  "resolutionNotes": null,
20  "comments": [],
21  "attachments": [],
22  "createdAt": "2026-04-28T22:54:12.3964",
23  "updatedAt": "2026-04-28T22:54:12.3964"
24 },
25 "timestamp": "2026-04-28T22:54:12.4990398"
26 }
```

: Authentication/Security & Notifications

Description:

Developed the user management module to handle system users and role-based access control. Implemented functionality for admins to view all registered users, update user roles (USER, ADMIN, TECHNICIAN), and manage access permissions. Integrated secure authentication using Google OAuth 2.0 and JWT tokens for protected API access and session validation. Also build up notification function, when approve or reject the booking, send notification to the user.

Endpoint Descriptions:

- **GET/oauth2/authorize/google:** Authenticate via Google OAuth2 and receive a JWT token
- **GET/api/auth/me:** Retrieve the currently authenticated user's profile
- **GET/api/auth/users:** List all registered users (Admin only)
- **PUT/api/auth/users/{id}/role:** Change a user's role (Admin only)
- **GET/api/notifications:** Retrieve all notifications for the authenticated user
- **GET/api/notifications/unread:** Retrieve only unread notifications
- **GET/api/notifications/unread/count:** Get count of unread notifications (for badge)
- **PATCH/api/notifications/{id}/read:** Mark a single notification as read
- **PATCH/api/notifications/read-all:** Mark all notifications as read
- **GET/api/notifications/preferences:** Retrieve per-user notification preferences
- **PUT/ /api/notifications/preferences:** Update per-user notification preferences (emailEnabled, pushEnabled)



Turn Your Ambition into *Achievement*

Empowering students with smart campus tools —
manage facility bookings, report incidents, and
track maintenance all in one unified hub.

[Resource Booking](#)[Incident Tickets](#)[Live Notifications](#)[Role Management](#)

99%

100+

4

24/7



SMART CAMPUS HUB

Welcome Back

Sign in to manage facilities, bookings &
maintenance at SLIIT. Only authorized university
members may access this system.

Continue with

[Sign in with Google](#)

or

[Continue with Facebook](#)[Get Started](#) →[Resource
Booking](#)[Incident
Tickets](#)[Notifications](#)[Role Access](#)

By signing in, you agree to SLIIT's terms of use.



Pradeep S Liyanage



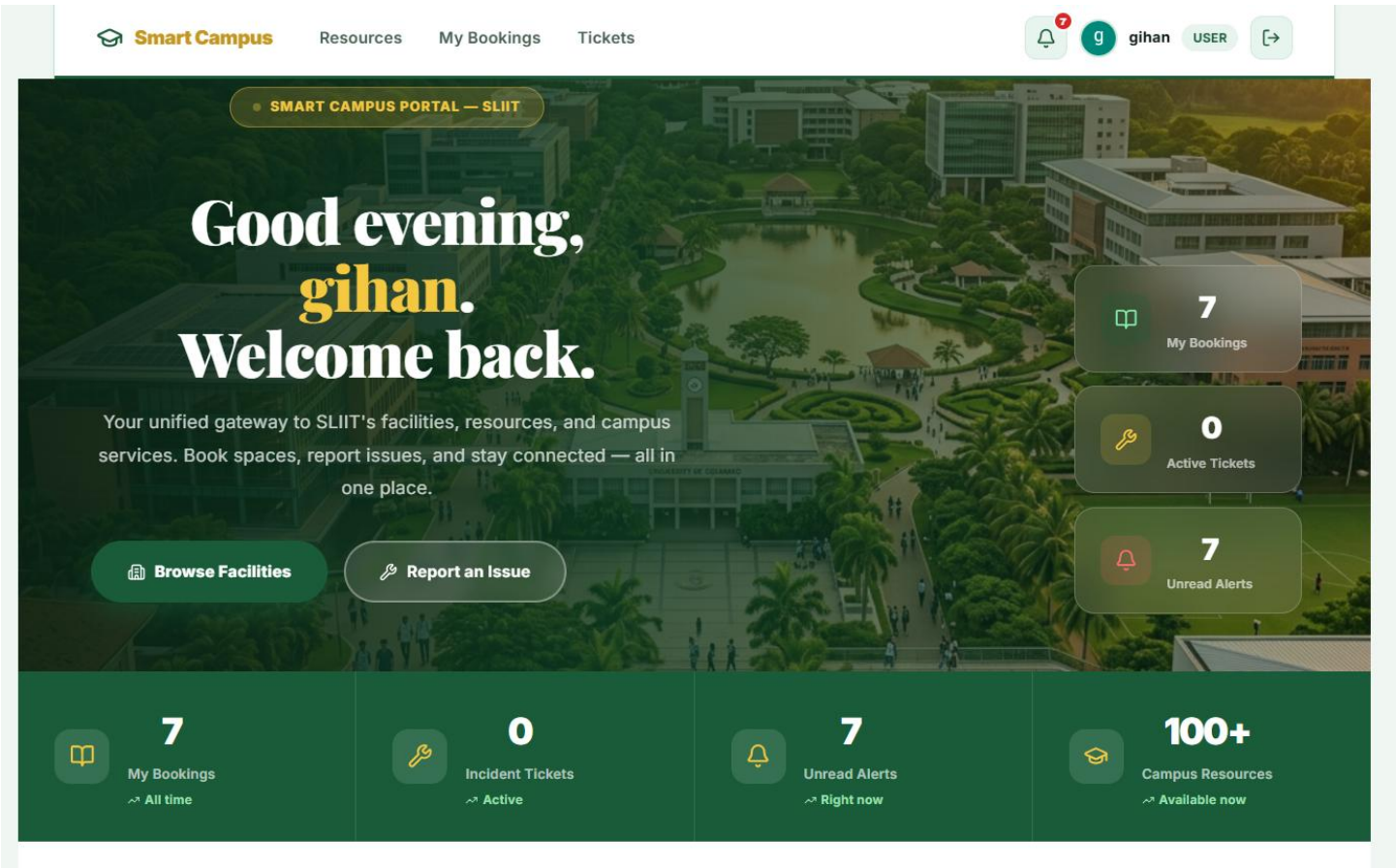
You previously logged in to smart-campus with Facebook.
Would you like to continue?

[Continue as Pradeep](#)

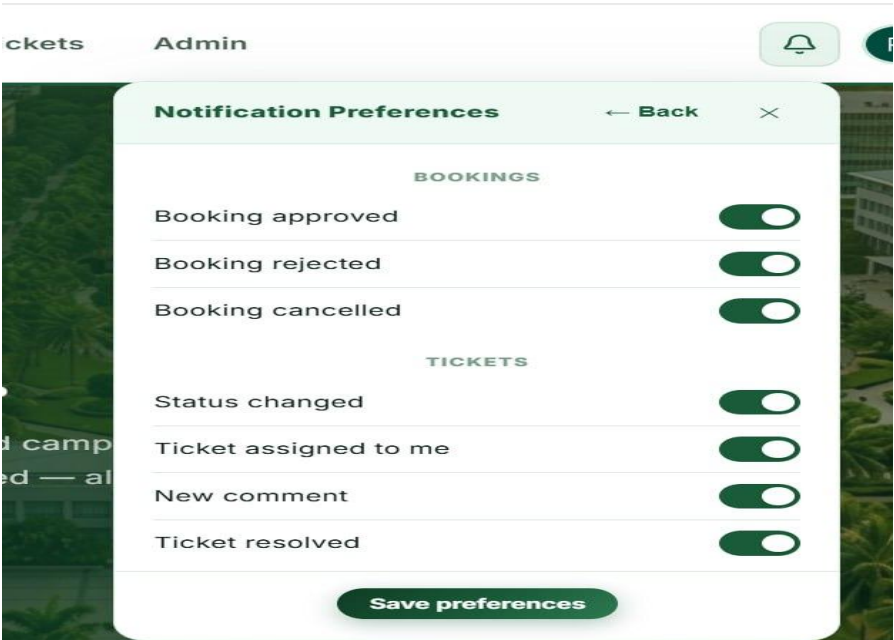
[Cancel](#)

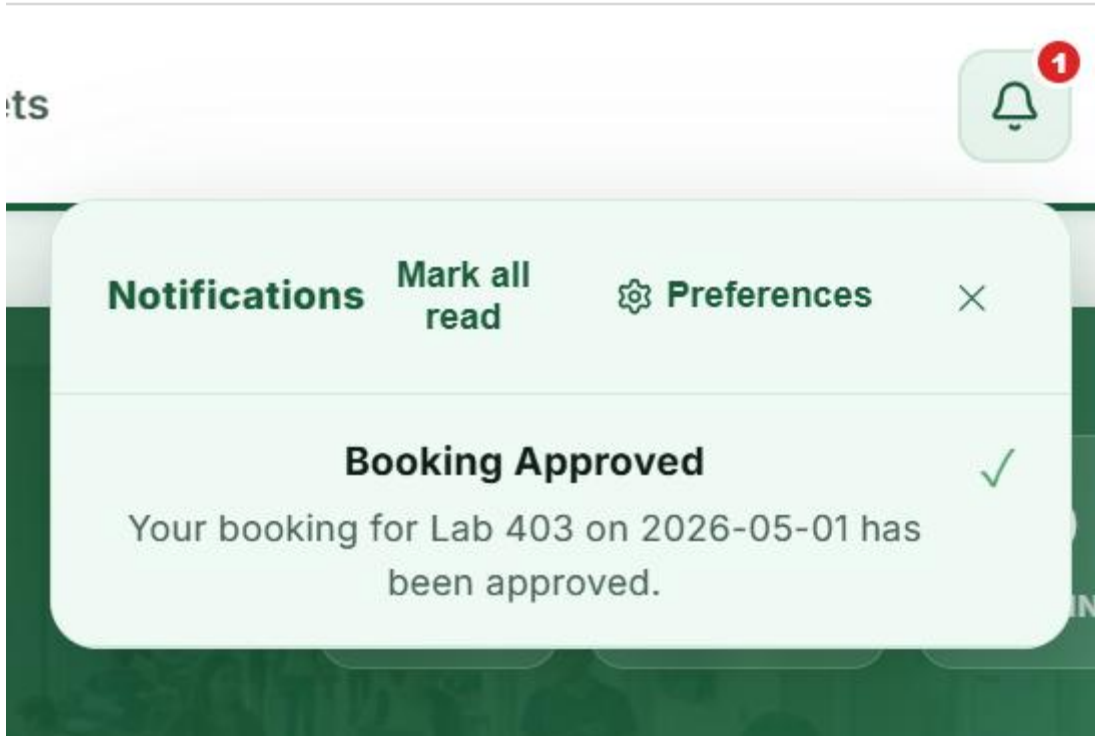
By continuing, smart-campus will receive ongoing access to the information that you share and Meta will record when smart-campus accesses it. [Learn more](#) about this sharing and the settings that you have.
[smart-campus's Privacy Policy](#)

Home Page:

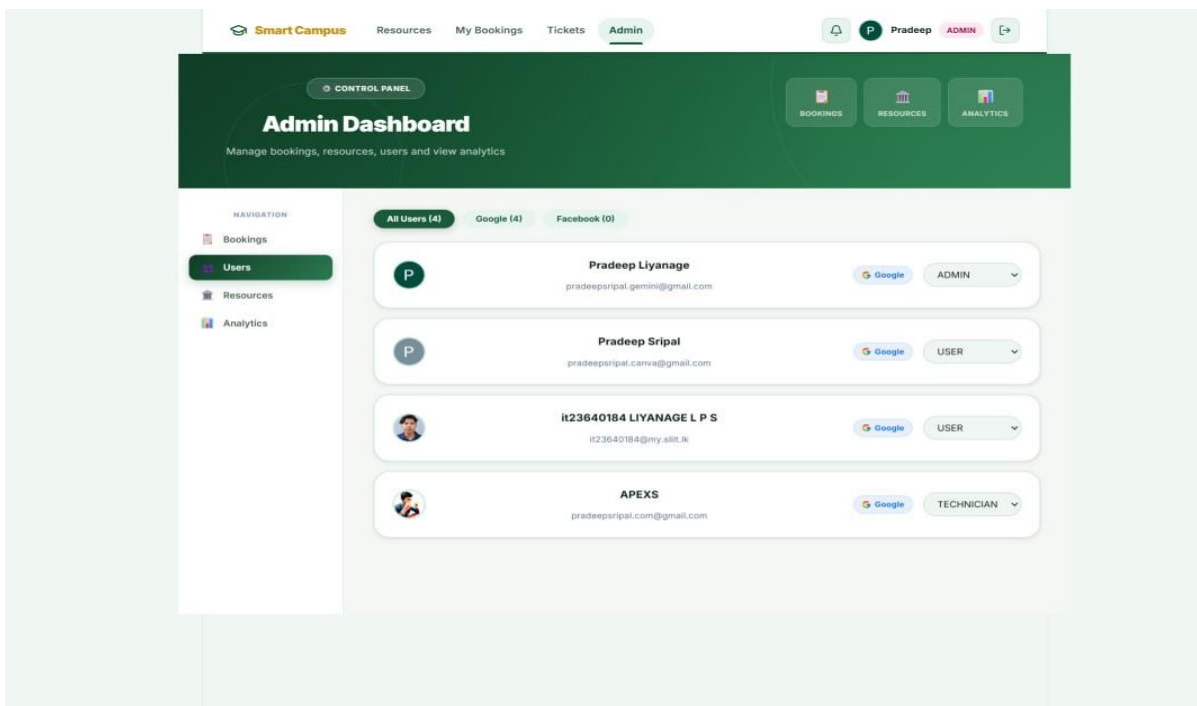


Notification :





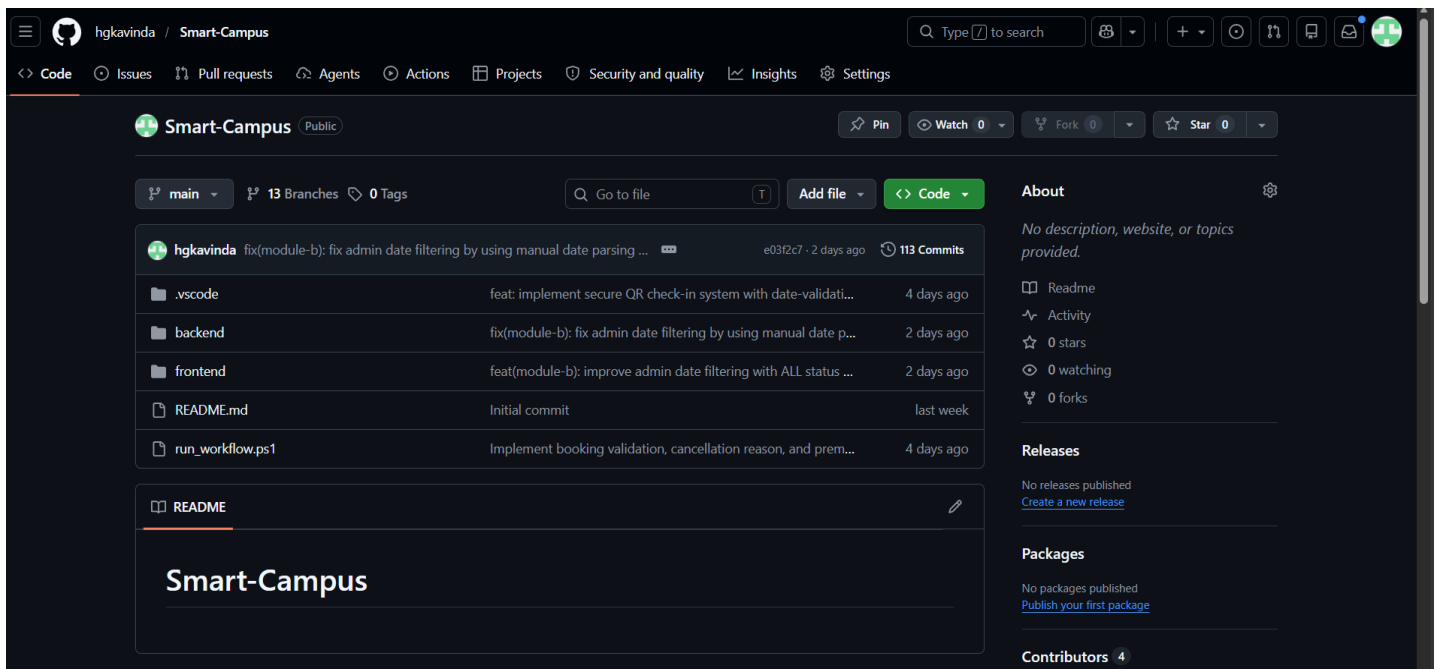
Admin side:



7. GitHub

- GitHub Repo link- <https://github.com/hgkavinda/Smart-Campus.git>
- OneDrive Link - https://drive.google.com/drive/folders/1OMm2o4XmOyfFJkl7PQpBTB95CMRh05nI?usp=drive_link
- Image of the GitHub Commit Tree

Git Repository :



All Branches :

🔍 search branches...

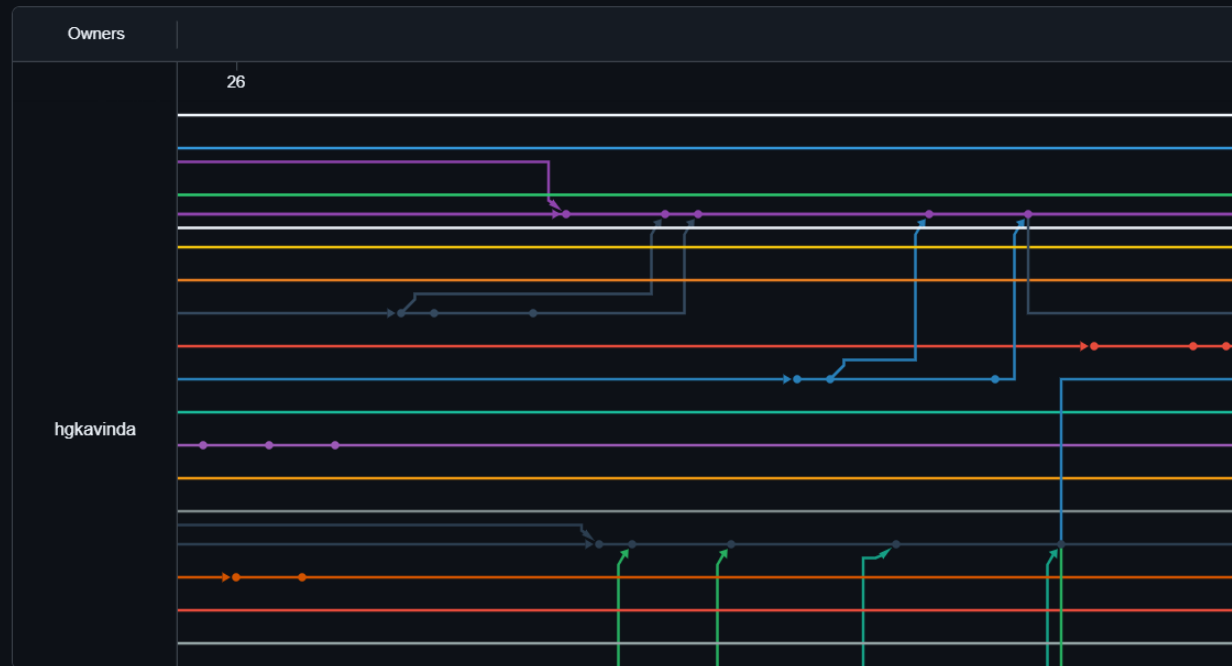
Branch	Updated	Check status	Behind	Ahead	Pull request	
main	2 days ago			Default		🗑️ ⋮
feature/module-b-booking-ui	2 days ago		113	102	#19	🗑️ ⋮
feature/module-a-resource-ui	2 days ago		113	26		🗑️ ⋮
feature/module-a-resource-backend	2 days ago		113	26		🗑️ ⋮
feature/module-b-booking-entity	2 days ago		113	95	#17	🗑️ ⋮
feature/module-de-admin-ui	3 days ago		113	87	#15	🗑️ ⋮
feature/module-d-notifications	3 days ago		113	65	#14	🗑️ ⋮
feature/module-e-auth-security	3 days ago		113	66	#13	🗑️ ⋮
feature/module-b-booking-backend	3 days ago		113	51	#16	🗑️ ⋮
feature/module-a-resource-entity	4 days ago		113	23	#7	🗑️ ⋮
feature/module-c-ticket-entity	4 days ago		113	9	#6	🗑️ ⋮
feature/module-c-ticket-ui	4 days ago		113	4	#5	🗑️ ⋮
feature/module-c-ticket-backend	4 days ago		113	4	#3	🗑️ ⋮

Commit Tree:



Network graph

Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.



8. References

- React.js Documentation. (n.d.). React – A JavaScript library for building user interfaces. Retrieved from <https://react.dev/>
- Spring Boot Documentation. (n.d.). Spring Boot – Build anything with Spring. Retrieved from <https://spring.io/projects/spring-boot>
- Java Persistence API (JPA). (n.d.). Jakarta Persistence Documentation. Retrieved from <https://jakarta.ee/specifications/persistence/>
- JSON Web Token (JWT). (n.d.). JWT.io – Introduction to JSON Web Tokens. Retrieved from <https://jwt.io/introduction/>
- MySQL Documentation. (n.d.). MySQL :: The world's most popular open source database. Retrieved from <https://dev.mysql.com/doc/>
- Axios HTTP Client. (n.d.). Axios – Promise based HTTP client for the browser and node.js. Retrieved from <https://axios-http.com/docs/intro>

- Tailwind CSS. (n.d.). Tailwind CSS Documentation – Rapidly build modern websites. Retrieved from <https://tailwindcss.com/docs>
- Google Gemini AI API. (n.d.). Google AI for Developers – Gemini API Documentation. Retrieved from <https://ai.google.dev/docs>
- GitHub Documentation. (n.d.). GitHub Docs – Version control and collaboration. Retrieved from <https://docs.github.com/>
- Mermaid.js. (n.d.). Mermaid – Diagramming and charting tool. Retrieved from <https://mermaid.js.org/>